Copyleaks

Analysis Report

Scan details

Scan Time	Total Pages	Total Words
May 11th, 2026 at 05:56 UTC	10	2388

AI Detection Report

2210991248._AmanSharma.docx

Plagiarism Detection

N/A

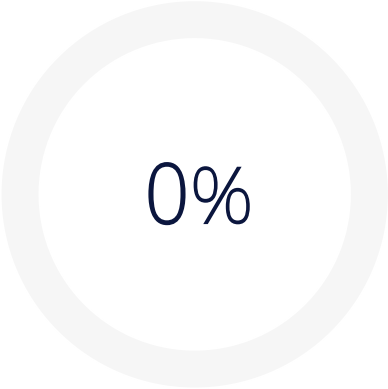
Plagiarism types	Text coverage
Identical	
Minor Changes	
Paraphrased	
Excluded	
Omitted Words	

AI Detection

0%

	Text coverage	Words
AI Text	0%	0
Human Text	100%	1
Excluded		
Omitted Words		1

AI Content



	Text coverage	Words
AI Text	0%	0
Human Text	100%	1
Excluded		
Omitted Words		1

About AI Detection

Our AI Detector is the only enterprise-level solution that can verify if the content was written by a human or generated by AI, including source code and text that has been plagiarized or modified. Learn more

AI Text

A body of text that has been generated or altered by AI technology. Learn more

Human Text

Any text that has been fully written by a human and has not been altered or generated by AI. Learn more

Copyleaks AI Detector Effectiveness

Credible data at scale, coupled with machine learning and widespread adoption, allows us to continually refine and improve our ability to understand complex text patterns, resulting in over 99% accuracy–far higher than any other AI detector–and improving daily. Learn more

Ideal Text Length

The higher the character count, the easier for our technology to determine irregular patterns, which results in a higher confidence rating for AI detection. Learn more

Reasons It Might Be AI When You Think It's Not

The AI Detector can detect a variety of AI-generated text, including tools that use AI technology to paraphrase content, auto-complete sentences, and more. Learn more

User AI Alert History

Historical data of how many times a user has been flagged for potentially having AI text within their content. Learn more

AI Logic

The number of times a phrase was found more frequently in AI vs human text is shown according to low, medium, and high frequency. Learn more

A Review of Artificial Intelligence Integration in Modern Software Development Practices

Aman Sharma
Department of computer science and engineering
Chitkara University Institute of Engineering and Technology
Rajpura, India
aman1248.be22@chitkara.edu.in

Akshat Khosla
Department of computer science and engineering
Chitkara University Institute of Engineering and Technology
Rajpura, India
akshat1226.be22@chitkara.edu.in

Dr. Shikha Tuteja
Department of computer science and engineering
Chitkara University institute of Engineering and Technology
Rajpura, India
shikha.1290@chitkara.edu.in

Aditya Aggarwal
Department of computer science and engineering
Chitkara University Institute of Engineering and Technology
Rajpura, India
aditya.1184be22@chitkara.edu.in

Aditya Chauhan
Department of computer science and engineering
Chitkara University Institute of Engineering and Technology
Rajpura, India
aditya1190.be22@chitkara.edu.in

Abstract - Artificial intelligence (AI) has revolutionized how developers write, test, and maintain code. Developers no longer need to code everything from scratch; instead, they employ sophisticated tools that can auto-complete code for them or rapidly identify and fix errors. In this paper, we investigate how AI, particularly data-driven, human language understanding AI apps, is interspersed throughout the software development lifecycle. In particular, we examine the use and usefulness of AI in coding, testing, debugging, and work efficiency. To evaluate AI use throughout software development, we performed a 2024-2026 literature review, studied current industry trends, and tested AI tools in real-world environments. We discovered AI coding assistants are effective at tedious, mind-numbing tasks. AI coding assistants speed development and testing and result in higher-quality code. By 2025, over 80% of professional dev teams were using AI tools, suggesting the rapid and widespread adoption of AI coding assistants. We still have hurdles towards widespread use, such as whether or not we trust AI recommendations, ethical considerations, and the reality that developers need to learn new skills. The rational position is that developers should be using AI as assistants, not replacing them.

Keywords: Artificial Intelligence, Machine Learning, Software Development, Automated Coding, Tech Trends, App Development, Coding Tools

I. INTRODUCTION

The software development practice has always improved as the engineers get better tools. From early systems to the modern software development platforms, each new generation of tools has accelerated the development process and led to higher quality software.

Now, the biggest leap in that journey is for artificial intelligence.I A I tools are no longer exclusive to researchers. Millions of developers around the world make AI part of routine day to day development. Smart code auto complete, code from natural English statements, auto bug fixes, these are now in the development tools developers already use, solutions now pervasive. The AI for software business is booming worth $674.3 million in 2024, projected to reach ~ $15.7B by 2033. Recent surveys show that more than the 76% of professional developers are already using AI tools, AI is no longer just a cool experiment. This paper explains how the AI is currently being used for the whole software development life cycle. It discusses how data-driven, language understanding solutions can be used for coding, automated testing, error prediction, bug fixes. It discusses the challenges facing developers and companies today, do we trust AI models completely, what's the ethical frontier we need to navigate, what skills programmers need to succeed in the AI-enabled workplace.

II. LITERATURE REVIEW

There have been a flurry of studies in recent weeks on how AI is shaping software development. These studies try to figure out everything from how new coding tools are tested and adopted by businesses to the technology behind them.

Initially, AI coding tools were very simple. They just predicted what the next line or word of code might be based on a few patterns, and that was not very useful. That all changed with the introduction of large language models. These advanced models were fed massive amounts of code, and so they can understand the context of a project and offer much more useful code.

AI is also transforming testing software. If a machine learning model has details about past errors, it can walk through a new program and look for where bugs might be likely before testing the code. That way, programming teams know where a new program might be prone to issues. This saves time and effort. Multiple studies find it's getting easier to figure out why a program is crashing. Clever debugging tools can sift through everything – error messages, system logs, developer notes – all at one time to figure out where a program went wrong. That's especially helpful for beginners, who can correct their own mistakes without having to keep

bothering senior developers. But researchers say the challenges are still significant.

There is concern that people cannot trust AI recommendations because copying AI-code without an understanding of it can immediately lead to security vulnerabilities. Others are concerned that new programmers will become too reliant on these new tools and never develop basic problem-solving skills. So for now, the consensus opinion is that AI should be used as a tool to speed up development, not replace humans.

III. METHODOLOGY

A. **Research Approach**

In this research, we conduct the literature survey on the current adoption of artificial intelligence to your software development. We gather the desired information from the research papers, industry reports and real-world case studies released between 2024 and the early of 2026. We selected these sources because it is up-to-date, well-sourced and focus on the recent AI coding methods.

B. **Research Scope**

Our research focuses on four dimensions, including the AI code generation, AI code testing, AI bug fixing and AI usability improvement for a developer in his day-to-day. For each dimension, we study both the underlying technology and the real-software-developers tools. We include the data from the tech.

C. **Surveyed Tools and Technologies**

In particular, we study the most popular AI coding assistants in use right now, such as GitHub Copilot, Amazon Code-Whisperer and Tabnine. We also review the intelligent testing frameworks and the emerging AI platform that can provide partial automation of some coding tasks. To measure the accuracy of these tools in reality, we collect the data from the public case studies and the reports released by the vendors .

D. **Evaluation Criteria**

To compare these tools and techniques fairly, we look at multiple criteria. The main aspects cover the productivity boost of the developer, the accuracy of the generated codes and the speed of the software development and testing. We also look at the speed of adoption of these tools to a typical team workflow and the spread of these tools in the industry. We also look at the ethical aspects, such as the bias in the AI model and the explainability of the model

IV. RESULTS AND FINDINGS

Our findings show that AI has dramatically changed how we build software from end to end. The biggest influence is that it automatically writes and completes code – a portion that accounts for almost 31.9% of the total AI spend and generates around $4.0 billion in just 2025. This means that developers are spending money on these tools – not to replace people but to save time in not typing out repetitive lines of code or constantly looking up online basic syntax and commands.

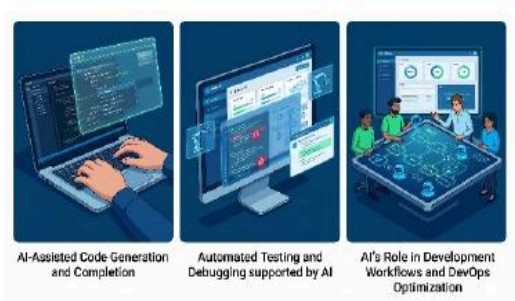
It's also making testing faster and more effective, as AI can determine which tests are "important" to run instead of having to run them all. The net result is that you can spend as little as 70% to 90% of the time on the build and test cycle for updates and the benefit that it's using data from older projects to determine where new bugs are more likely to pop up than with older testing methods.

Almost everyone seems to be on it – more than 80% of software teams were using at least one AI tool by 2025 – and spending money on large models like Anthropic's Claude, meaning that some are spending money on these tools – not to replace people but to save time – as AI is no longer a fad, but rather part of the job.

Table 1: How AI Affects Different Parts of Development

Domain	How AI is used	Impact
Writing Code	Autocomplete and code suggestions	31.9% market share
Testing	Selecting the right tests	Run 70-90% faster testing times
Fixing Bugs	Reading and understanding error messages	Largely less time spent hunting down bugs
Usuage	Enterprise AI development tools	80% of teams use them by 2025

Figure 1: Illustrations of how AI is used to generate code, test, and team processes.



But even with all the advantages, some issues remain. For starters, sometimes the AI does not explain its solutions, so what developers don't know is why it proposed that particular code snippet. Another issue is the data it needs. Machine learning requires a large amount of historical data to learn from, and small or new projects often do not have that. Last, and probably most importantly, it can be challenging to get these very new AI tools to work with older, legacy computer systems, which can be a tough sell for businesses.

V. THE INVINCIBLE FORCES DRIVING AI ADOPTION

There are not many big forces that are driving software teams to start adopting AI at such a breakneck speed. The big driver is that developers are pressured to do more with less. Software is becoming more and more complex to build. And schedules are becoming more and more aggressive. AI tools take over the boring repetitive work. They program and flag errors early and that saves a lot of time for programmers and for the businesses that they serve. Another big driver are how big the large language models have become. They have learned from huge amounts of code. So they are very fluent in a lot of programming languages and configurations. And they are really useful for daily coding work and not just in test environments.

Culture in the workplace is also critical. So we see the teams that have bosses that encourage experimentation and learning that are the ones that succeed with AI. The teams that don't have that culture are fighting to make AI work. There are also things that prevent certain industries from adopting AI. Things about code security, who owns code that the AI writes, and the fear of becoming lazy and dependent are all big things. And that is especially true for regulated industries like banking, health care, and government.

VI. CONCLUSION

This review has shown that AI is no longer a nice-to-have experiment. It has become a must in software development these days. From writing code, to running tests, to debugging, it's true that AI is helping teams move faster and produce better quality software when used properly.

The key must is that AI is there to help developers, not replace them.

These tools are great for doing the boring, repetitive typing work. That frees devs up to do the harder, more complex, more hard-to-code things of software dev, like designing systems and working out logic puzzles. That's where companies get the most bang for the buck.

Because these tools are such a rapidly moving target, companies just can't ignore them if they want to keep their competitive edge. But, buying an AI tool is not a magic bullet either. They need to be setting up policies as to how to use the AI-generated code, they need to keep educating their employees, and they need to be especially careful about the serious security, fairness, and transparency issues. It's the teams that take the time to think about how to integrate AI into their work that will have the most benefit but with the fewest problems.

VII. FUTURE WORK

There is still plenty of work to be done in the combining coding and AI space, especially. The next major trend is the shift from the hinting/autocomplete models we see today to more autonomous smart AI agents that do more of the work themselves. These models will not just give hints on how to write something but be more like teammates that will write code that can be directly used in the project.

We will also shift to more programming in plain English. Soon, you should be able to just type out what you want an app to do and the AI will give you the full python code. This is going to be massive for beginners to learn from, and also for expert programmers to be able to produce a prototype version of their app ideas much faster.

Beyond writing code, AI will also be heavily involved in the "DevOps" side of things, like launching updates, managing servers, and doing all the telemetry on how well the system is running under the hood. As AI improves, the barrier between writing code and managing servers will become invisible.

All of this is pretty amazing, but there are also a lot of difficult ethical problems that will need to be solved. Researchers should be actively working to ensure that these are fair, that user data is private, and that they are fully "explainable" in terms of how exactly they came to their solution. We also need to figure out who is responsible if the AI writes bad code. Remember, this is the only way people will trust AI enough to use it all the time in the future.

VII. REFERENCES

[1] Grand View Research, "Artificial Intelligence in Software Development Market Size, Share & Trends Analysis Report", Grand View Research, San Francisco, CA, 2024.

[2] Menlo Ventures, "The State of Generative AI in the Enterprise," Menlo Ventures Industry Report, 2025.

[3] GitHub, "GitHub Copilot: Your AI Pair Programmer," GitHub Inc., 2024. [Online]. Available: https://github.com/features/copilot

[4] Amazon Web Services, "Amazon Q Developer (formerly CodeWhisperer) Product Overview," AWS, 2024. [Online]. Available: https://aws.amazon.com/q/developer/

[5] T. B. Brown et al., "Language models are few-shot learners," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 1877–1901.

[6] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[7] M. Kim, S. Lee, and J. Park, "Automated Test Case Generation Using Machine Learning Techniques: A Review," IEEE Transactions on Software Engineering, vol. 50, no. 2, pp. 415-430, 2024.

[8] Anthropic, "Claude Model Usage and Impact in Enterprise Environments," Anthropic Research, 2025.

[9] Stack Overflow, "2025 Developer Survey: AI Tools Adoption and Usage," Stack Overflow, 2025.

[10] A. Vaswani et al., "Attention is all you need," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017, pp. 5998–6008.